Prediction-Error-Triggered Memory Updates for Language

Do learned update times capture hierarchical structure?

Esra Genc

Research note — initial results, July 2026 | `GitHub repository`

Background. Language does not change in the same way at every character or token. Some input only continues a local pattern, while other input introduces a new word, opens a clause, changes an entity, or forces the reader to revise an interpretation. Conventional language models nevertheless usually update dense internal representations at every step.

Research question. I am testing a model in which prediction-error-conditioned evidence accumulates until it triggers an update to persistent memory. The central question is whether the timing of these learned updates captures useful information about linguistic and hierarchical structure, rather than simply producing a regular sparse update schedule.

Related work. Earlier models have divided recurrent computation across fixed timescales, learned latent boundaries, or learned to skip state updates [1, 2, 3]. Recent byte-level models also allocate more computation to less predictable regions of text [4]. The present experiment focuses on a narrower question: whether prediction error can drive hard memory-update events whose timing is structurally useful and supports generalisation beyond the training regime.

Tested method. The model reads raw bytes. A local recurrent state processes every byte, while three persistent memories operate at fast, intermediate, and slow timescales. Each memory has an integrate-and-fire clock. The clock receives the current context and prediction error, accumulates update pressure, and emits a hard event when its threshold is crossed. A memory changes only when its event fires; otherwise it preserves its previous state.

Main idea. *Let prediction errors accumulate, and update each persistent memory only when the model has gathered enough evidence that its state should change.*

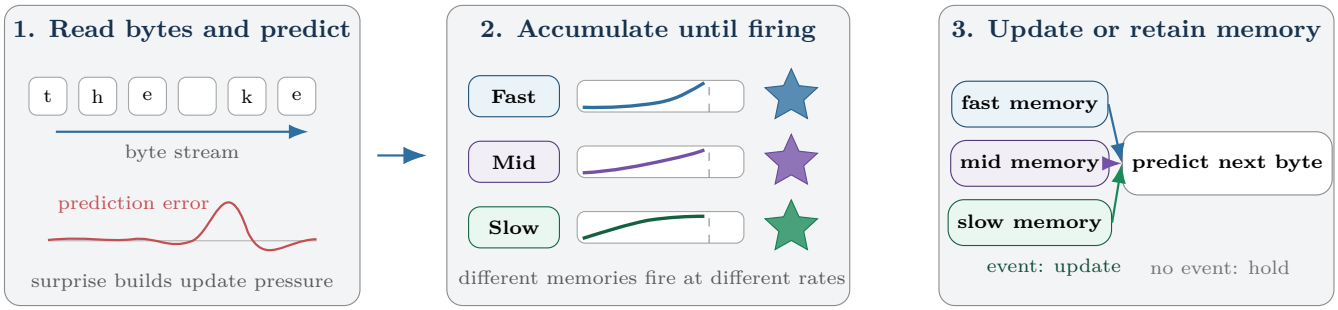

Figure 1: **Concept of the tested model.** The model predicts the next byte and uses the resulting prediction error as part of the evidence supplied to three integrate-and-fire clocks. Each clock accumulates evidence until it emits an event. That event updates one persistent memory; between events, the memory is kept unchanged.

Controlled experiment. To isolate hierarchical structure, I trained the model on byte sequences with three kinds of nested brackets and neutral filler words. Training sequences had a maximum nesting depth of four. I then evaluated the model on held-out sequences at familiar depths and at unseen depths of six and eight.

The learned scheduler was compared with an otherwise matched model whose fast, mid, and slow memories updated periodically at the same approximate rates. Both models were selected using validation closing-bracket loss. The main structural task was predicting the correct closing bracket, which requires the model to retain information about currently open brackets. Lower NLL means better predictions.

Current results.

Test split	Learned close NLL	Periodic close NLL	Learned accuracy (%)	Periodic accuracy (%)	Interpretation
Familiar depths ≤ 4	1.188	1.152	47.6	46.3	Periodic has better likelihood; learned is slightly more accurate.
Unseen depth 6	1.610	1.616	45.0	42.6	Results are close, with an accuracy advantage for learned timing.
Unseen depth 8	1.807	1.885	40.7	37.1	Learned timing is better on all main structural metrics.

At unseen depth eight, learned timing reduced closing-bracket negative log-likelihood by approximately **4.1%** and

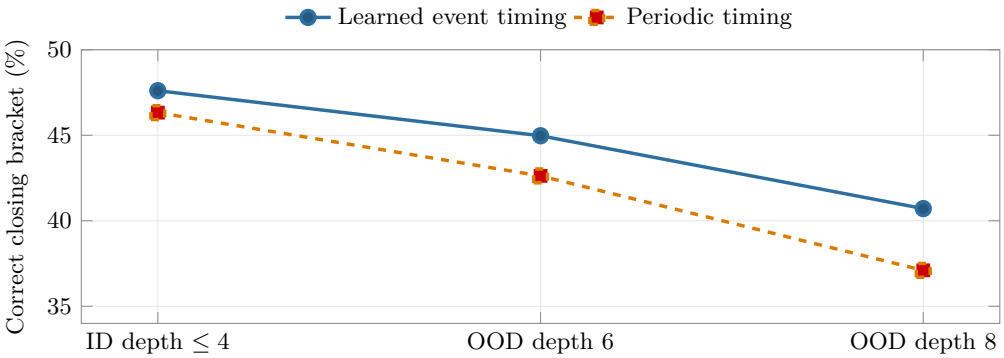

Figure 2: **Closing-bracket accuracy as nesting depth increases.** The periodic model is competitive on familiar structure, while the model with learned prediction-error-conditioned timing loses less accuracy at unseen nesting depths. Results are from one training seed.

improved accuracy by **3.6 percentage points** relative to periodic scheduling. It also had lower overall byte loss and lower loss on structural symbols. The pattern is that periodic updates are more efficient on familiar structures, while learned timing appears more robust as the hierarchy becomes deeper than anything seen during training.

Do the events correspond to structure? The slowest memory fired on only about 3.2% of positions, but its events were **2.30 times enriched at opening brackets** and **1.72 times enriched at any bracket transition**. Push and pop labels were not used for training. This suggests that the rare slow events developed a preference for moments when new nested structure begins. The effect is selective rather than complete: the slow channel detected only a minority of all opening operations, and its alignment with closing operations was weak.

Timing intervention result. Within the learned model, shifting the mid-timescale event sequence worsened closing-bracket prediction, and a periodic replacement was slightly worse. However, random and fully ablated mid schedules were not reliably worse on the closing-bracket metric. The clearest current evidence therefore comes from the out-of-depth comparison and the slow-channel alignment.

What the current experiment supports.

- Prediction-error-conditioned integrate-and-fire clocks can train with hard, sparse updates at several timescales.
- Learned scheduling does not automatically improve in-distribution prediction over a fixed sparse clock.
- In this one-seed experiment, learned scheduling generalized better to substantially deeper hierarchical structure.
- A rare slow event stream developed unsupervised alignment with structure-opening operations.

Limitations. This is an initial controlled experiment, not yet a claim about natural language. The result comes from one random seed and a synthetic bracket task. The model also contains a local recurrent pathway that updates at every byte, which may allow it to solve parts of the task without using the event memories. The next evaluation should repeat both learned and periodic models across at least three seeds, test stronger rate-matched baselines, and then move to natural-language tasks with irregular state changes, such as entity tracking, long-distance agreement, and syntactic revision.

Conclusion. The current evidence supports a modest but useful hypothesis. Learned memory updates may be less advantageous for familiar local patterns, yet more robust when a sequence requires deeper hierarchical generalisation. The next question is whether the same timing mechanism discovers meaningful update points in natural language, where lexical, syntactic, semantic, and discourse states change at irregular moments.

References

[1] J. Koutník, K. Greff, F. Gomez, and J. Schmidhuber, "A Clockwork RNN," *Proceedings of the 31st International Conference on Machine Learning*, 2014.

[2] J. Chung, S. Ahn, and Y. Bengio, "Hierarchical Multiscale Recurrent Neural Networks," *arXiv preprint arXiv:1609.01704*, 2016.

[3] V. Campos, B. Jou, X. Giro-i-Nieto, J. Torres, and S.-F. Chang, "Skip RNN: Learning to Skip State Updates in Recurrent Neural Networks," *arXiv preprint arXiv:1708.06834*, 2017.

[4] A. Pagnoni, R. Pasunuru, P. Rodriguez, et al., "Byte Latent Transformer: Patches Scale Better Than Tokens," *arXiv preprint arXiv:2412.09871*, 2024.